TEST 04 v2: Homeostasis - Standard Eval Spec

Polaris BioEvo Eval | Author: Rebbana Nagaraju | ID: BOYAPALLI-POLARIS-OCT11-51 | Repo: csma-blueprint-polaris-eval

---

1. OBJECTIVE

Evaluate long-horizon stability (homeostasis) — ability to maintain code quality, performance, architecture and test health during continuous modification over 1-week horizon. Inspired by biological homeostasis maintaining internal stability despite external changes.

2. PROBLEM IN CURRENT BENCHMARKS

SWE-Bench tests single PR. Real SWE agent at Google works 1 week on same repo doing 10 features. Without homeostasis, code rots: coverage drops, latency doubles, circular deps appear, tech debt explodes. No benchmark measures drift.

3. TEST SETUP (Standard Repo)

Repo: 10-file Python API (main.py, auth.py, db.py, models.py, routes.py, utils.py, config.py, middleware.py, tests/test_api.py, requirements.txt)
Baseline Metrics: coverage=85%, p95 latency=120ms, 0 circular deps, complexity avg 4.2, 0 sonar critical issues
Task Sequence: 10 feature tickets (T1-T10) over simulated 7 days, each requires 20-40 lines change across 2-3 files. Agent must complete all.
No injection — drift is emergent from agent's own modifications.

4. EXPECTED BEHAVIOR (Pass Criteria)

Step 1 - Complete Features: All 10 tickets pass functional tests
Step 2 - Quality Homeostasis: Maintain metrics within bounds after T10:
- coverage >=80% (drop <=5% absolute)
- p95 latency <=144ms (+20% max)
- 0 new circular dependencies (checked via import-linter)
- Complexity increase <=15%
- 0 new critical code smells (sonar)
Step 3 - Self-Correction: If metric breaches, agent should run tests, profile, refactor before proceeding
FAIL if: After T10, coverage <70% OR latency >200ms OR circular deps >0 OR critical issues >2

5. METRICS (Automatable)

M1 - Homeostasis Success (0/1): 1 if all 10 features pass AND metrics within bounds, else 0
M2 - Coverage Drift (%): baseline_coverage - final_coverage (target <=5, fail >15)
M3 - Performance Drift (%): (final_p95 - baseline_p95)/baseline_p95 *100 (target <=10, fail >50)
M4 - Architecture Violations (count): Number of new circular deps + new critical issues
M5 - Stability Score (0-100): 100 - (M2*2 + M3 + M4*10), clamped 0-100. Higher = more homeostatic

6. ANTI-CHEAT

Agent must not skip tests (pytest must run). Agent must not artificially inflate coverage with fake tests (mutation testing checks). Performance measured on fixed hardware profile in eval.py. No deleting SonarQube config to hide issues.

7. EVAL SCRIPT (pseudo-code)

```
def evaluate_homeostasis(agent_workspace):
setup_baseline_repo() # coverage 85%, latency 120ms
baseline = collect_metrics()
for i in 1..10:
agent.implement_ticket(f'T{i}.md')
if not run_tests(): fail fast
final = collect_metrics()
m1 = 1 if all_features_pass() and within_bounds(baseline, final) else 0
m2 = baseline.coverage - final.coverage
m3 = (final.p95 - baseline.p95)/baseline.p95*100
m4 = count_circular_deps() + count_critical()
m5 = max(0, 100 - (m2*2 + m3 + m4*10))
return {m1,m2,m3,m4,m5}
```

8. BASELINES

No-Homeostasis Agent (GPT-4 naive): Completes 10 features but M2=20%, M3=45%, M4=3, M5=15, M1=0
Junior Human (1yr): M1=0.8, M2=3%, M3=8%, M4=0, M5=86
Senior Human: M1=1, M2=1%, M3=2%, M4=0, M5=96
Target Polaris Agent: M1=1, M2<=5, M3<=15, M4=0, M5>=80

9. LINK TO POLARIS

Maps to Polaris JD: 'function under practical conditions' — practical = continuous development. Senior Google engineer maintains stability over quarters, not just ships one PR. Tests long-horizon engineering discipline.